SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I N Saptha Nageswar (192372001) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

ANSWER SHEET - Assessment Tool 2 (Data Interpretation)

Course: Internet Programming (CSA43) **CO-1** **Total Marks:** 20

Question 1: HTTP Response Analysis

1. Incorrect or invalid fields

- **Content-Length: -150** - invalid. Per RFC 9110/9112, Content-Length must be a non-negative integer (number of bytes in the body). A negative value is illegal.
- **Date: Mon, 01 Apr 2026 ...** - inconsistent. 1 April 2026 falls on a **Wednesday**, not Monday. The day-name must match the calendar date in the HTTP-date format, so this is a malformed/incorrect value.
- **Content-Type: text/html** - syntactically valid, but no charset is given (better: `text/html; charset=UTF-8`); this is a recommendation, not an error.
- **Status line and Connection: keep-alive** - valid. (keep-alive is the default in HTTP/1.1, so it is redundant but harmless.)

2. Issue with Content-Length and its impact

Content-Length tells the client exactly how many bytes of body to read, and on a persistent (keep-alive) connection it marks where this response ends and the next begins. A negative length cannot be interpreted, so message framing breaks. Impact:

- The browser/client treats the response as malformed and may reject it with an error (e.g. ERR_INVALID_CONTENT_LENGTH) or show a blank page.
- On a keep-alive connection the client cannot tell where the body ends, so the connection gets out of sync and later responses may be misread.
- Proxies/caches may reject or mishandle it; inconsistent length handling can also enable request/response smuggling attacks.
- Progress indicators and partial-download handling fail.

3. Does the status code match the response correctness?

No. 200 OK says the request succeeded and the representation is delivered correctly, but the response itself is malformed (invalid Content-Length, wrong Date). The status code is syntactically valid, but it does not reflect the real state of the response. If the server failed while generating the body, it should return **500 Internal Server Error**; if it is healthy, it must send 200 with correct headers.

4. Corrections

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
Connection: keep-alive
```

Here 150 is only an example: the value must equal the actual size of the body in bytes (or use `Transfer-Encoding: chunked` when the size is not known in advance, and never send both headers together).

Question 2: DOM Structure Analysis

1. Missing or improperly closed elements

- **<p>** is never closed - the closing `</p>` is missing; it is only closed implicitly when `</div>` appears.
- The text **HelloWorld** and the title **TestPage** have no space (probably meant 'Hello World' / 'Test Page').

- No `<!DOCTYPE html>`, no `<meta charset>` and no `lang` attribute on <html> (not errors for the parser, but required for a standards-compliant page).

2. How the browser interprets it

HTML parsers are error-tolerant. When the parser meets `</div>` while a <p> is still open, it applies the *implied end tag* rule and closes the <p> first. The resulting DOM is:

```
html
 +-- head
 |    +-- title ("TestPage")
 +-- body
      +-- div
           +-- p ("HelloWorld")
```

Because there is no DOCTYPE, the browser switches to **quirks mode**. If the same file were parsed as XHTML/XML, it would fail with a **well-formedness error** because XML does not allow unclosed tags.

3. Impact on rendering and layout

- In HTML mode the text still shows as one paragraph inside the div, so the visible result is almost unchanged.
- Quirks mode can change the box model, default font sizes, table/line-height behaviour and margins, giving inconsistent layout across browsers.
- If more content were added after the unclosed <p>, block elements would silently close it and CSS/JS selectors could target an unexpected structure.
- In strict XML/XHTML mode nothing would render - only an error page.

4. Corrected DOM

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Test Page</title>
</head>
<body>
  <div>
    <p>Hello World</p>
  </div>
</body>
</html>
```

Question 3: GET vs POST in Login System

1. Differences in data transmission

Aspect	GET	POST
Where data goes	Appended to the URL as a query string (/login?user=admin&pass=1234)	Sent inside the HTTP request body
Visibility	Visible in address bar, history, bookmarks, server/proxy logs, Referer header	Not shown in the URL (but not encrypted by itself)
Size limit	Limited by URL length (about 2-8 KB depending on browser/server)	Practically no limit; supports large data and file uploads
Purpose / semantics	Retrieve data; safe and idempotent	Submit data / change server state
Caching and replay	Can be cached, bookmarked and re-sent by refresh	Not cached by default; browser warns on resubmission

2. Which method suits login, and why

POST is the suitable method. Login credentials are sensitive and the request changes state (creates a session), which matches POST semantics. POST keeps the username and password out of the URL, browser history, bookmarks and most access logs, and is not cached or replayed by accident.

3. Security risks of GET for authentication

- Credentials are exposed in the browser history, address bar and shoulder-surfing.
- Web server, proxy and CDN logs store the full URL, including the password in plain text.
- The Referer header can leak the URL (with credentials) to third-party sites and analytics.
- URLs can be bookmarked, shared or cached, so credentials spread; links are also open to CSRF-style abuse.
- Even over HTTPS the URL stays in logs and history, so HTTPS alone does not fix it.

4. Best practices for secure login

- Use **POST over HTTPS/TLS** only; redirect HTTP to HTTPS and set HSTS.
- Store passwords hashed with a salt using bcrypt, scrypt or Argon2 - never in plain text.
- Use CSRF tokens, secure session cookies (`Secure; HttpOnly; SameSite`) and regenerate the session ID after login.
- Limit login attempts / add rate limiting, account lockout or CAPTCHA; add multi-factor authentication.
- Validate and sanitise input (prevents SQL injection/XSS) and give generic error messages ('Invalid username or password').
- Set `autocomplete` and `type="password"` properly and never log credentials.

Question 4: HTML Code Inspection

1. Missing closing tags

- `</h1>` - the <h1>Welcome heading is never closed.
- `</p>` - the paragraph 'This is a paragraph' is never closed.

(<html>, <head>, <title> and <body> are closed properly. The DOCTYPE is present.)

2. Issues with the <img> tag

- <img> is a **void element**, so it needs no closing tag in HTML5 (in XHTML it must be self-closed: `<img ... />`) - this is not an error in HTML5.
- The **alt** attribute is missing - required for accessibility (screen readers) and for display if the image fails to load; also affects SEO and validation.
- No **width/height** given, so the layout can shift while the image loads (reflow); the image path is relative, so it fails if image.jpg is not in the same folder.
- Because the <img> sits inside the unclosed <p> (which sits in the unclosed <h1>), it ends up inside the heading structure.

3. How browsers handle these errors

The HTML5 parsing algorithm recovers silently instead of showing an error. A <p> start tag does **not** automatically close an open <h1>, so the paragraph and the image become children of the h1. Everything is closed implicitly when </body> is reached:

```
body
 +-- h1 "Welcome"
      +-- p "This is a paragraph"
           +-- img (image.jpg)
```

Result: the paragraph text is rendered in heading style (large and bold), the heading hierarchy and semantics are wrong, and the layout differs from what the author intended. Validators will flag it, and strict XHTML parsing would stop with an error.

4. Corrected code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph</p>
  <img src="image.jpg" alt="Sample image" width="300" height="200">
</body>
</html>
```

Question 5: Browser-Server Communication Flow

1. Interpretation of each step

Step	What happens	Typical delay
1. User enters URL	Browser parses scheme, host, path; checks HSTS, cache and history	Negligible
2. DNS lookup	Domain name is resolved to an IP address (browser/OS cache, then resolver, root/TLD/authoritative servers)	Tens to hundreds of ms on a cache miss
3. HTTP request sent	TCP three-way handshake (and TLS handshake for HTTPS), then request line + headers (GET /index.html) are sent	1-3 round trips (network latency)
4. Server processes	Web server routes the request, runs application code, queries the database, builds the response	Depends on server load, code and DB speed
5. Response returned	Status line, headers and body (HTML) travel back to the client	Bandwidth and body size
6. Browser renders	Parses HTML to DOM, CSS to CSSOM, fetches CSS/JS/images, builds layout and paints	Render-blocking CSS/JS, many sub-requests

2. Where delays can occur

- **DNS lookup:** slow or distant resolver, no cached entry.
- **Connection setup:** TCP and TLS handshakes over a high-latency link.
- **Server processing:** heavy computation, slow database queries, overloaded server (high TTFB).
- **Transfer:** large unoptimised responses, low bandwidth, packet loss.
- **Rendering:** render-blocking CSS/JS, large images, many extra requests and redirects.

3. Role of DNS

DNS works as the internet's phonebook. Browsers need an IP address to open a connection, but users type human-readable names. DNS translates the domain (e.g. www.example.com) into an IP address through a hierarchy: browser/OS cache, recursive resolver (ISP or public), root server, TLD server (.com) and authoritative name server. Without a successful DNS lookup, no HTTP request can be sent (the 'server not found' error). It also enables load balancing and CDN routing to a nearby server, and its results are cached according to the record's TTL.

4. Optimisations to improve performance

- **DNS:** use fast DNS providers, DNS prefetch (`<link rel="dns-prefetch">`), sensible TTLs, fewer third-party domains.
- **Network:** HTTP/2 or HTTP/3, keep-alive, TLS session resumption, preconnect, use a CDN to be closer to users.
- **Server:** caching (server, Redis, browser Cache-Control/ETag), database indexing, efficient code, load balancing.
- **Payload:** gzip/Brotli compression, minify HTML/CSS/JS, compress and lazy-load images (WebP/AVIF).
- **Rendering:** reduce render-blocking resources (async/defer scripts, critical CSS inline), reduce redirects and the number of requests.

Conclusion

Interpreting the headers, markup and request flow shows that well-formed, standards-compliant HTML and correct HTTP headers are essential: browsers hide many mistakes through error recovery, but the result can still be wrong rendering, quirks mode, failed responses or security problems. Using POST over HTTPS for logins and optimising DNS, connection, server and rendering stages gives a secure, reliable and fast web experience.